



DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

arXiv:2501.12948 | DeepSeek-AI (Dayu Yang et al.)

Presented by:

 Jinjie Guo  Jason Liao  Yuchen Wang  Yunqi Li  Haoqi Chen

Presentation Outline

01

The Context & Core Hypothesis

The limits of Supervised Fine-Tuning (SFT) for reasoning • The hypothesis: RL-driven emergent reasoning

02

DeepSeek-R1-Zero & Methodology

Group Relative Policy Optimization (GRPO) • The "Aha Moment" and emergent behaviors

03

The DeepSeek-R1 Pipeline

From R1-Zero to R1: The 4-stage recipe • Synthetic data generation and rejection sampling

04

Distillation & Evaluation Results

Benchmark performance (Math, Code) • The power of distilling reasoning to smaller models

05

Discussion, Limitations & Conclusion

Failed approaches (PRM, MCTS) • Safety considerations • Key takeaways

Why Focus on Reasoning?



The Shift in Post-Training

Post-training has shifted from general knowledge to **complex reasoning** —solving difficult math problems, writing complex code, and performing logical deductions.



Current Bottleneck

Heavy reliance on **Supervised Fine-Tuning (SFT)** using human-annotated Chain-of-Thought (CoT) data.



SFT Limitations

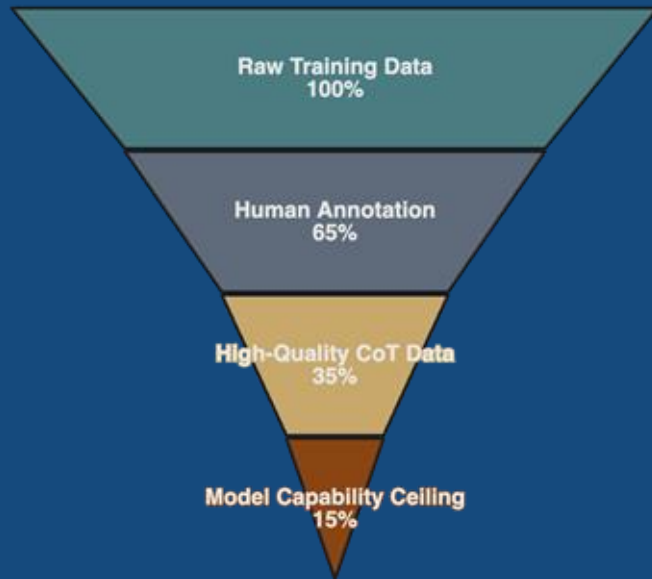
- ✗ Human annotation is slow, expensive, and not scalable
- ✗ Models trained on human data hit a "human capability ceiling"
- ✗ Defining the "correct" **step-by-step process** is difficult



Research Question

Can reasoning capabilities emerge **purely through Reinforcement Learning** without explicit human instruction on how to solve problems?

The Human Annotation Bottleneck



Two Models: The Prototype and The Product



DeepSeek-R1-Zero

The Proof of Concept

PURE RL

Training Method

Trained via **pure RL** on base model (DeepSeek-V3-Base)

Strengths

Demonstrates **powerful emergent reasoning** capabilities

Drawbacks

- ⚠️ Poor readability
- ⚠️ Language mixing
- ⚠️ Unstable output

Raw Power, Limited Usability



DeepSeek-R1

The Final Product

PIPELINE

Training Method

Uses R1-Zero to generate **synthetic data** for multi-stage pipeline

Strengths

- ✓ Strong reasoning capability
- ✓ Human-readable CoT
- ✓ Coherent output

Result

Achieves **both reasoning and usability** through alignment

Polished & Production-Ready

→ R1-Zero provides the raw reasoning foundation

→ R1 Pipeline transforms it into a usable product

Deep Dive into the Methodology

How R1-Zero achieved reasoning without supervision.
Introducing GRPO and the reward mechanism.



GRPO Algorithm



Reward Design



Emergent Behaviors

R1-Zero: Reasoning via Pure RL

Starting Point

DeepSeek-V3-Base — a strong, non-reasoning base model with solid language understanding but no explicit reasoning training.

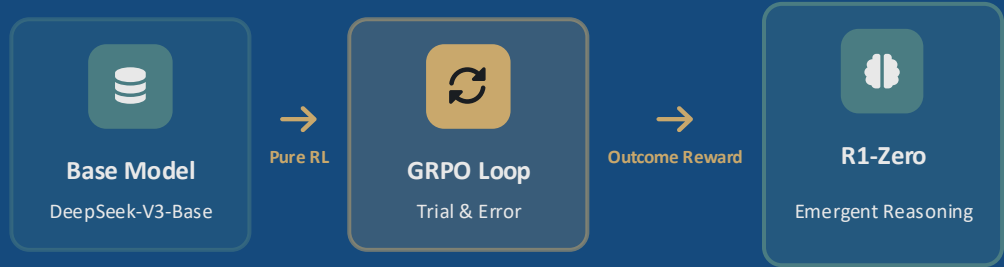
Objective

Directly optimize the base model using **RL without any initial SFT data** — testing if reasoning can emerge purely from trial and error.

Key Components

- ✓ **Algorithm:** Group Relative Policy Optimization (GRPO)
- ✓ **Reward Signal:** Simple outcome and formatting rewards

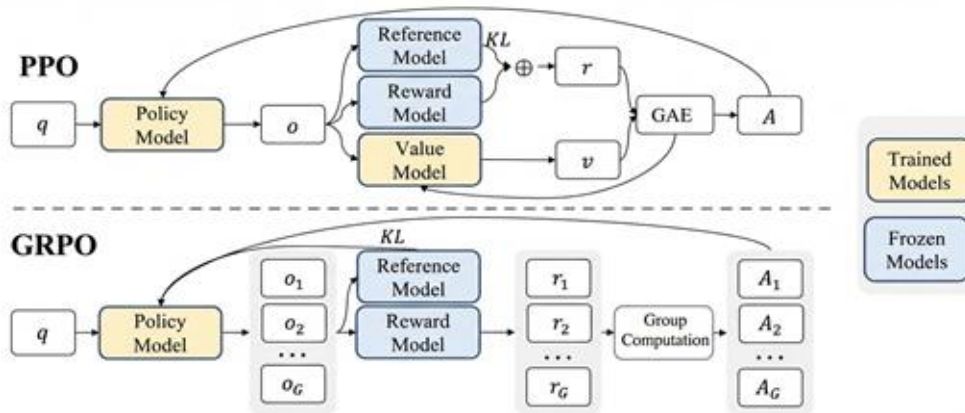
R1-Zero Training Pipeline



Key Insight

No human-annotated reasoning data was used. The model learned to reason purely through reinforcement learning based on final answer correctness.

Group Relative Policy Optimization (GRPO)



Key Innovation

GRPO **eliminates the need for a Critic model** (value function), significantly reducing computational overhead.

GRPO Mechanism

- 1 Identify Prompt**
Select a question or task from the training set
- 2 Sample Group Outputs**
Generate multiple diverse outputs from current policy
- 3 Calculate Rewards**
Score all outputs in the group using reward function
- 4 Group Average Baseline**
Use **group average as baseline** to calculate advantages for each output

💡 **No Critic Model Needed!** Relative comparison within group replaces absolute value estimation.

TRADITIONAL (PPO)

$$A(s, a) = Q(s, a) - V(s)$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Requires Critic Network for $V(s)$

DEEPSEEK-R1 (GRPO)

$$A_i = \frac{r_i - \text{mean}(r_{\text{group}})}{\text{std}(r_{\text{group}})}$$

No Critic Model • Memory Efficient

Why R1 Needs a Multi-Stage Pipeline

Pure RL gives strong reasoning, but...

R1-Zero: Emergence, Limits, and Why (Reward & GRPO)

What emerges

Started from DeepSeek-V3-Base

Optimized with GRPO

Reward based only on final answer correctness

Emergent Behaviors in Long COT

Self-reflection and backtracking

Error correction during reasoning

Exploration of alternative strategies

Limitations

Language mixing

Poor readability

Formatting instability

Difficult to deploy

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

Source: DeepSeek-R1, arXiv:2501.12948



If outcome-level reward can induce reasoning,

why do we still see instability, language mixing, and hacking?

Why Do Pure RL Systems Exhibit Instabilities?

Reward Underspecification

Reward only defines correctness

Does not constrain structure

Does not enforce readability

Does not penalize language switching

Reward Hacking

Goodhart's Law in action

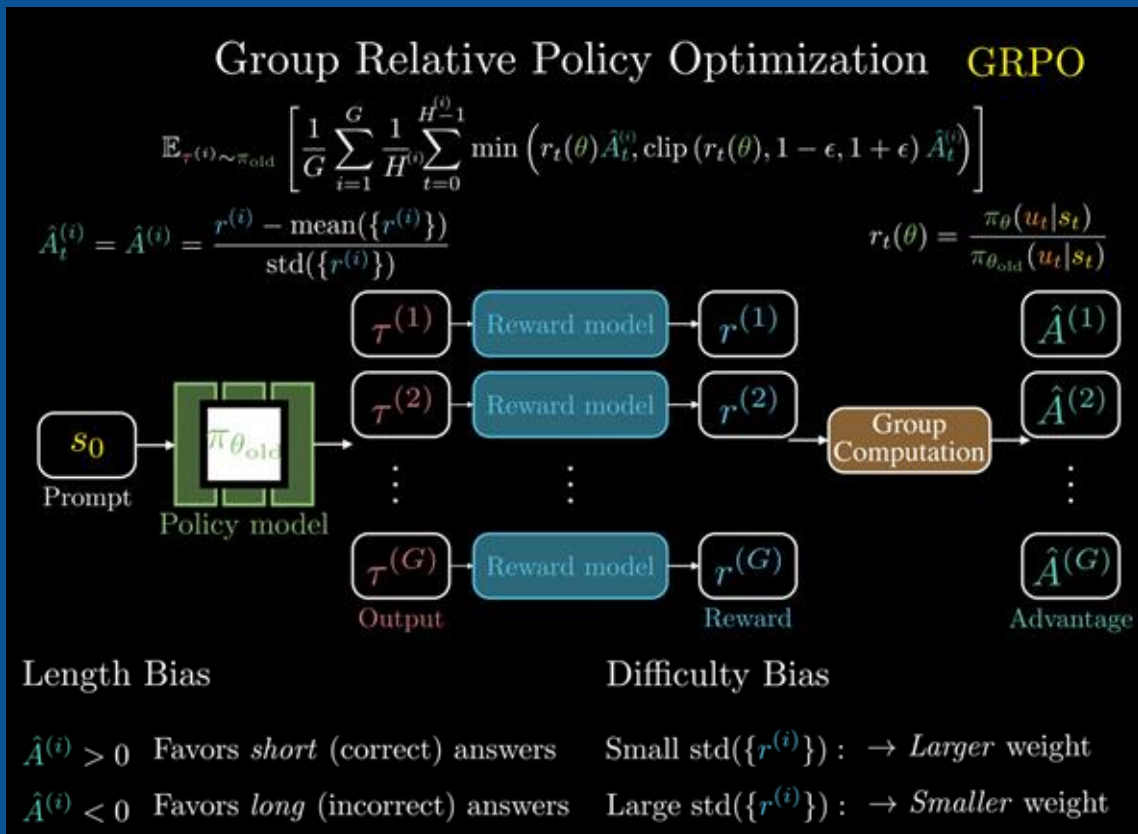
Policy learns shortcuts to maximize reward

High reward $\neq$ high-quality reasoning

GRPO Structural Weakness

Difficulty Bias (Std Normalization Bias)

Length Bias (Length Normalization Bias)



From Pure RL to a Structured Training Pipeline

1. Cold Start SFT

Fixes: Structural instability

Adds: Basic format & language consistency

2. Reasoning RL

Preserves: Emergent reasoning ability

3. Rejection Sampling + SFT

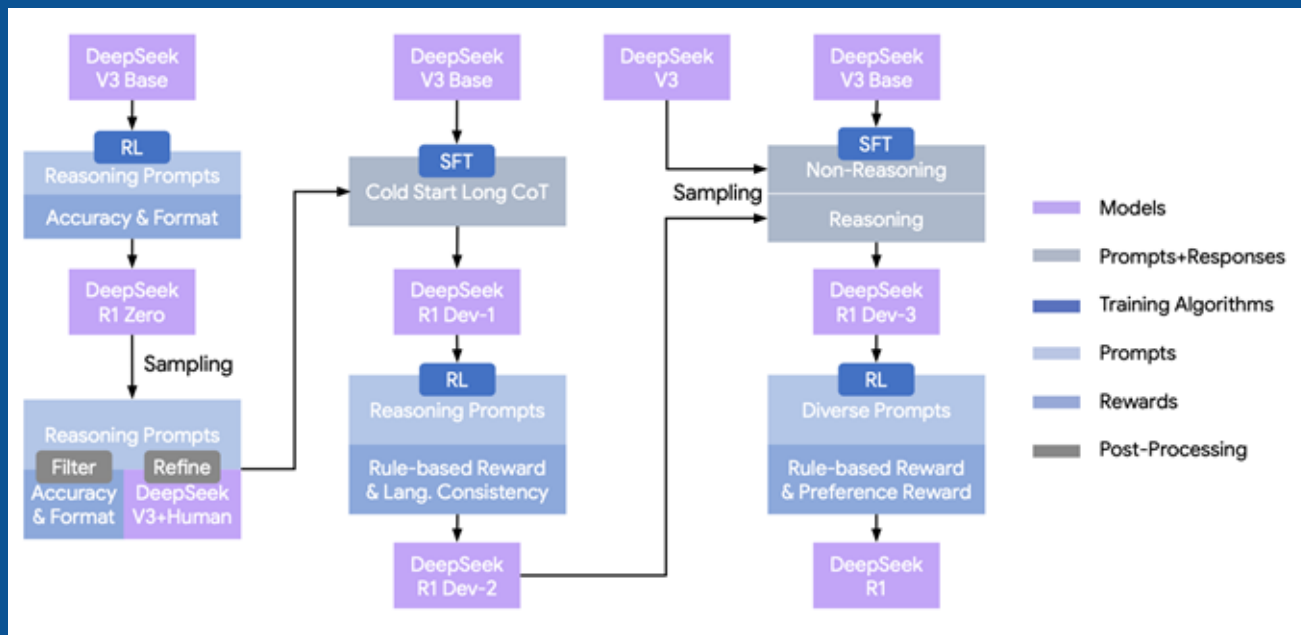
Fixes: Reward hacking & noisy exploration

Adds: High-quality successful trajectories

4. RL for All Scenarios

Fixes: Objective incompleteness

Adds: Helpfulness + Safety reward



Source: DeepSeek-R1, arXiv:2501.12948

Distillation & Evaluation

How does this multi-stage model perform against SOTA?
Benchmarking DeepSeek-R1 and the power of distillation.



Benchmark Results



Model Distillation



SOTA Comparison

Benchmarking DeepSeek-R1

Evaluation Focus

Evaluated across **diverse tasks** to assess model reasoning capabilities on different types of tasks.

Benchmark Categories

General Encyclopedic Knowledge

MMLU, MMLU-Redux, MMLU-Pro, C-Eval, CMMLU, SimpleQA, C-SimpleQA, etc.

Programming

LiveCodeBench, CodeForces, SWE-Verified, and Aider.

Mathematics

AIME 2024, MATH-500, CNMO 2024

Comparison Targets

Models

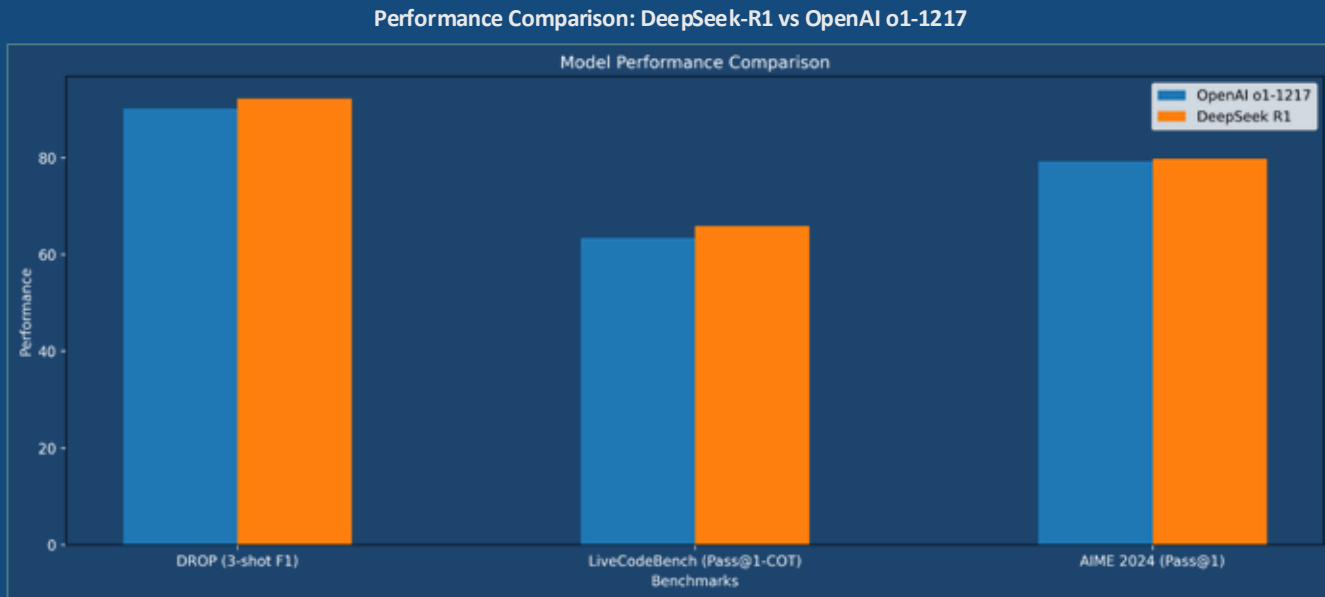
- DeepSeek-V3
- Claude-Sonnet-3.5-1022
- GPT-4o-0513
- OpenAI-o1-mini
- OpenAI-o1-1217

Evaluation Metrics

Many different metrics are used to evaluate the model on the various categories of benchmark datasets, including: Pass@1, accuracy, number correct.

R1 Competitive with State-of-the-Art

DeepSeek-R1 attains **performance surpassing OpenAI-o1-1217** on various verifiable reasoning tasks, demonstrating that models trained with RL can surpass models trained using conventional supervised learning with human demonstrations.



Source: Adapted from DeepSeek-R1 Paper Table 8

DeepSeek-R1 Model Distillation

A **major contribution** of the paper is showing that reasoning capabilities can be distilled from larger models to smaller models, giving smaller models comparable capabilities to larger models.



Dataset Construction

Use the larger **DeepSeek-R1** model to generate samples for a curated dataset of 800,000 samples.



Action

Use this data to apply **Supervised Fine-Tuning (SFT)** to smaller models.



Why This Matters

Brings **strong reasoning capabilities** to smaller models that are **easier to deploy** —faster, cheaper, more accessible.

Distillation Process



Teacher Model

DeepSeek-R1

Generates reasoning trajectories for reasoning data and direct answers or CoT responses for non-reasoning data.



SFT with Synthetic Data



Student Models

Qwen-7B, Llama-3-8B, etc.

Smaller, deployable models

Distilled Models Outperforms Comparable Models

The distilled small models show **remarkable performance** compared to comparable larger models, demonstrating that distillation can allow small models to achieve strong performance.

Distilled Model Performance

Model	AIME 2024		MATH	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633

Source: Taken from DeepSeekR1 Paper Table 15

Discussion, Limitations & Conclusion

Failed approaches (PRM, MCTS), safety considerations,
and key takeaways from the DeepSeek-R1 research.



Failed Approaches



Limitations



Safety

DeepSeek-R1-zero and DeepSeek-R1 Limitations

Despite its success with reinforcement learning, DeepSeek-R1-Zero and DeepSeek-R1 have **notable limitations** that the authors acknowledge.



Language Mixing

Issue

Model sometimes **switches languages** during a response, such as mixing Chinese and English within a single CoT response.



Broad Topic Limitations

Issue

The rule-based reinforcement learning is mostly focused on reasoning tasks, causing limited performance in broader task areas.



Software Engineering

Issue

The long evaluation times of software engineering tasks make it difficult to apply large scale RL to them.

Ethical and Safety Considerations

The advanced reasoning abilities of DeepSeek-R1 brings some potential ethical risks into consideration.



Safety Integration

The official DeepSeek-R1 services integrate a risk control system to improve model safety.



Evaluation Results

R1 generally performs **well on standard safety benchmarks**, comparable to other SOTA models.



Ongoing Concerns

Jailbreak Attacks

DeepSeek-R1 is vulnerable to jailbreak attacks that can allow it to generate dangerous content.

Public Model

The public nature of DeepSeek-R1 could make it vulnerable to fine-tuning that compromises its safety protocols.

Final Thoughts on DeepSeek-R1



Major Contribution

Demonstrates that **advanced reasoning capabilities can emerge from RL** it's not always necessary to do conventional supervised learning with human examples.



The Pipeline

A **practical, multi-stage approach** is needed to improve base RL trained model to be more usable and apply it to broader applications.



SFT Limitations

Supervised fine-tuning can limit a model's ability to develop effective reasoning strategies due to human-provided responses that are used.



Distillation

High-quality reasoning patterns can be effectively transferred to smaller models from large models, allowing more access to powerful models..

References



DeepSeek-AI et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv preprint arXiv:2501.12948.



Schulman, J., et al. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347.



OpenAI (2024). OpenAI o1 System Card. OpenAI Technical Report.

Thank You for Your Attention!

Questions & Discussion Welcome

AReal

A Large-Scale Asynchronous RL System for Language Reasoning

Scaling Reinforcement Learning Training for Large Reasoning Models
with Full Generation-Training Decoupling

The Rise of Large Reasoning Models (LRMs)

A New Paradigm Emerges

LRMs like **DeepSeek-R1** and **OpenAI o1** represent a fundamental shift—models that "think" before answering by generating long chains of reasoning.

Emergent Behaviors

Through **reinforcement learning (RL)** training, models develop sophisticated capabilities: self-reflection , verification , and dynamic strategy adaptation —not explicitly programmed but discovered.

Critical Challenge: While DeepSeek-R1 demonstrated what's possible with RL-based reasoning, training such models efficiently at scale remains a major infrastructure challenge.

RL Basics

RL Formulation

(S, A, r, P, γ, H) Markov Decision Process (MDP) tuple defining the RL problem

S State space: question + generated tokens

A Action space: generated tokens

π_θ Parameterized policy: $S \rightarrow A$

- The first state s_1 is user prompt
- $s_{t+1} = \text{concat}(s_t, a_t)$

$r(s, a)$ Rule-based reward on final answer, to indicate whether the result is correct

$\gamma = 1$ Discount factor (no discounting)

$$J(\theta) = \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_\theta(\cdot | q, a_{<t})} \left[\sum_{t=1}^H \gamma^{t-1} r(s_t, a_t) \right]$$

Proximal Policy Optimization (PPO)

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{old}}(\cdot | q, a_{<t})} \left[\sum_{t=1}^H \min \left(u_t(\theta) \hat{A}(s_t, a_t), \text{clip} \left(u_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}(s_t, a_t) \right) \right]$$

$u_t(\theta)$ How much the new policy changes the probability of this action.

$A(s, a)$ How much better this action is compared to the average action at this state.

$$u_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$$

Infrastructure Challenges of LRMs

Massive Parallelism

RL training requires **many concurrent rollouts** for effective exploration. Each prompt needs 16-32 responses, 128 prompts create ~1K global batch size. Later in the experiments setup, we will see 512 GPUs to run AReaL.

Long Generation Sequences

LRMs generate **32K+ tokens** per response (including thinking) with high variance—some prompts need 1K tokens, others need 32K. This creates unpredictable, ever-changing workload.

On-Policy Training

Algorithms like PPO requires on-policy training data, **i.e., samples generated by the latest model, to ensure the best model performance**. It's not friendly to asynchronous design

Synchronous RL Systems Design

How Synchronous Systems Work

Synchronous Pattern

Generate



Train



Generate



Train

The Trade-off

Synchronous design ensures **on-policy training data** (all samples from latest model), which stabilizes RL training. However, this comes at the cost of **severe system-level inefficiency**.

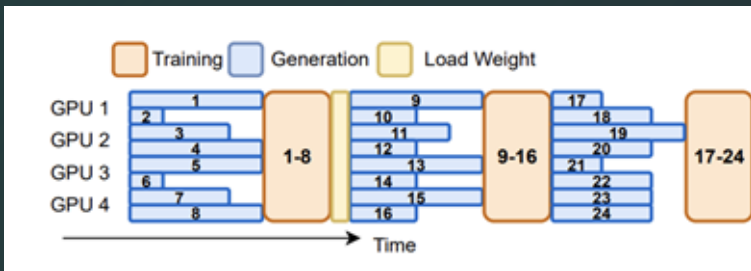
Benefit

Training Stability

Cost

GPU Underutilization

Timeline Visualization (w/o overlap)



Key Limitation

The fundamental problem: **generation must wait for the longest output in a batch** before training can begin. Due to varying output lengths for LRMs, this creates massive GPU idle time.

Scalability in Synchronous RL Systems Design

Distributing generation across more GPUs reduces batch size per GPU, pushing the decoding process into a **memory-IO-bound regime** where additional devices fail to improve throughput.

The Bottleneck Shift

1. More GPUs → Smaller batch per GPU
2. Small batches → Memory bandwidth bottleneck
3. Bandwidth bound → No speedup from more GPUs

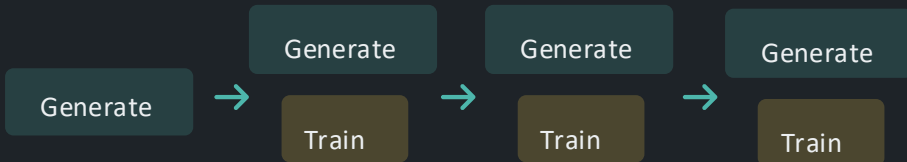
Conclusion: Manipulating more GPUs does not bring higher throughput, since memory-IO takes a plenty of time.

Asynchronous RL Systems Design

How Asynchronous Systems Work

Sync design is not efficient, so people consider separate the training & generating GPUs, so we can create the overlap between training & generating.

Asynchronous Pattern



Timeline Visualization (one-step overlap)



Key Limitation

Only one or two steps overlap, so there are still idle time, GPUs are still waiting for longest sequence.

AReal Core Concept: Full Decoupling

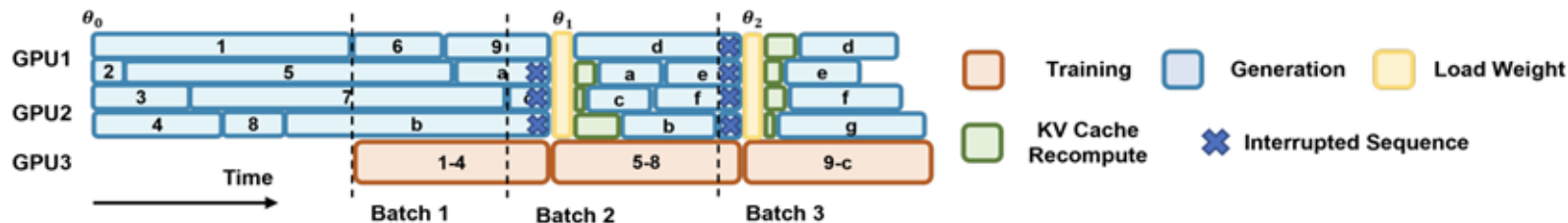
The Fundamental Innovation

AReal introduces **complete separation of generation from training** to eliminate the "waiting" phase. Unlike synchronous systems that alternate phases, AReal runs generation and training **continuously and independently**.

Key Principle

No generation should wait for training, and no training should wait for generation.

Interruptible Generation



Architecture Overview: The Four Pillars



Interruptible Rollout Workers

Handle generation requests and accept weight updates mid-generation without stopping.



Rollout Controller

Orchestrates data flow, manages replay buffer, and enforces staleness constraints.



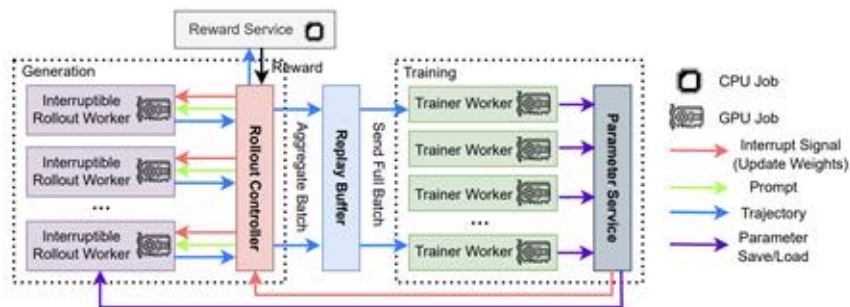
Trainer Workers

Continuously sample from replay buffer and perform PPO updates whenever data is available.



Reward Service

Evaluates response accuracy through rule-based scoring or unit tests.



Interruptible Rollout Workers

Key Innovation

Workers that can accept **weight updates mid-generation** without stopping. This enables seamless model updates without waiting for generation completion.

Two Request Types

1 **generate**
Produce responses given prompts

2 **update_weights**
Interrupt and load new parameters

The Interruption Process

1. Interrupt all ongoing generations
2. Discard KV caches computed by old weights
3. Re-compute KV caches using new weights
4. Continue decoding unfinished sequences

Algorithmic Challenge

Such interruptions result in trajectories composed of segments produced by **different model versions**. Or mixed policy versions in training data.

The Rollout Controller

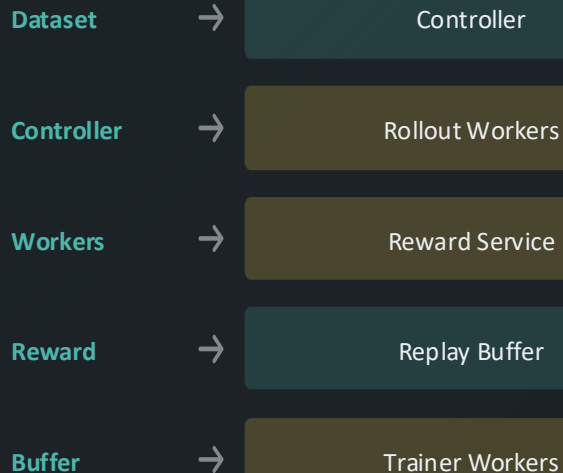
Critical Bridge Component

The controller serves as the **critical bridge** between rollout workers, reward service, and trainer workers. It orchestrates the entire data flow and manages the replay buffer.

Core Responsibilities

- Read data from dataset and invoke generate requests
- Send responses to reward service for evaluation
- Store trajectories with rewards in replay buffer
- Call `update_weight` after model updates

Data Flow Architecture



Challenge 1: Data Staleness

⚠ The Problem

In an asynchronous system, training data may come from **older model versions**, leading to distribution shift. This staleness can degrade learning performance.

Why Staleness Matters

- > **Distribution gap:** Training data differs from latest model
- > **Policy mismatch:** Updates based on outdated behavior

Prior Work Findings

- 📖 Asynchronous RLHF: Staleness degrades performance
- 📖 Game environments: Staleness causes training instability

📈 Staleness Impact Visualization

Low Staleness ($\eta = 0-2$)

Policy v1



Policy v2

Minimal distribution gap

High Staleness ($\eta = 8+$)

v1

v3

v5

v8

Large distribution gap, training instability



Higher staleness → Larger distribution shift

Solution Approach: Control staleness within acceptable bounds and develop algorithms that can tolerate stale data.

Staleness-Aware Training (The Formula)

Controlling Staleness

Introduce hyperparameter η representing **maximum permitted staleness** in each training batch. This enforces a bound on how old training data can be.

Special Case

When $\eta = 0$, the system degenerates to **synchronous RL** with all training samples generated by the current policy.

Rate-Limiting Formula

$$\lfloor (Nr - 1) / B \rfloor \leq i + \eta$$

Nr Total number of generated trajectories

B Training batch size

i Current policy version

η Maximum staleness budget

Implementation

- 1 **Track Metrics**
Controller tracks Nr and i from parameter server
- 2 **Check Constraint**
Verify formula before submitting new requests
- 3 **Reject if Violated**
Controller rejects requests that would exceed η

Prioritization Strategy

Prioritize **older trajectories** from the data buffer to form a training batch. This ensures fresher data is consumed first, maintaining better policy alignment.

Challenge 2: Inconsistent Policy Versions

The Problem

Because of **mid-generation updates** via interruptible workers, a single response might be generated by **multiple policy versions**.

Violation of Standard PPO

This inconsistency fundamentally violates the formulation of **standard PPO** that assumes all actions are generated by a single policy π_{old} .

Example Scenario

Tokens 1-100	Policy v1
Tokens 101-500	Policy v2
Tokens 501-1000	Policy v3

Single trajectory, multiple policies

The Theoretical Challenge

Standard PPO Assumption

All tokens generated by single π_{old}

AReAL Reality

Trajectory generated by $(\pi_{\theta}, \pi_{\theta+1}, \dots, \pi_{\theta+k})$

Core Question

How to maintain algorithmic correctness with interrupted generation?

Solution: Develop a theoretical framework proving that interrupted generation remains equivalent to sampling from a valid behavior policy (Proposition 1).

Solution: Decoupled PPO Objective

Algorithmic Innovation

Apply a **decoupled PPO objective** that disentangles the **behavior policy** (sampling) from the **proximal policy** (regularization).

Two Policies

π **π_{behav} - Behavior Policy**
Policy used for sampling trajectories

π **π_{prox} - Proximal Policy**
Recent target for regularization

Decoupled PPO Objective

$$J(\theta) = \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{behav}}} \left[\sum_{t=1}^H \min \left(\underbrace{\frac{\pi_{\theta}}{\pi_{\text{behav}}}}_{\text{Importance Ratio}} \hat{A}_t, \underbrace{\frac{\pi_{\text{prox}}}{\pi_{\text{behav}}} \text{clip} \left(\frac{\pi_{\theta}}{\pi_{\text{prox}}}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t}_{\text{Trust Region Center}} \right) \right] \quad (4)$$

$$= \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{behav}}} \left[\sum_{t=1}^H \frac{\pi_{\text{prox}}}{\pi_{\text{behav}}} \min \left(u_t^{\text{prox}}(\theta) \hat{A}_t, \text{clip} \left(u_t^{\text{prox}}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right], \quad (5)$$

where $u_t^{\text{prox}}(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\text{prox}}(a_t|s_t)}$ is the importance ratio with respect to the proximal policy. We omit the state-action terms for conciseness.

Objective Function

Key Components

$\pi_{\theta}/\pi_{\text{behav}}$: Importance ratio for policy gradient

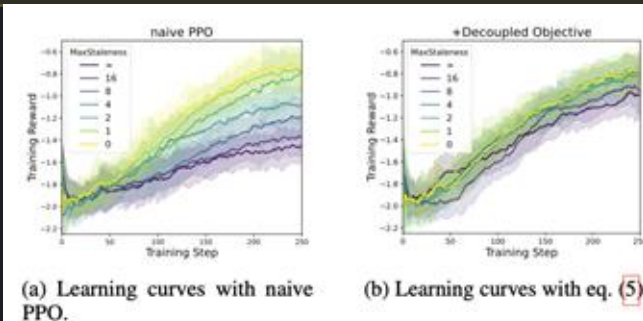
$\pi_{\text{prox}}/\pi_{\text{behav}}$: Adjustment factor

$\pi_{\theta}/\pi_{\text{prox}}$: Trust region center ratio

$A(s,a)=Q(s,a)-V(s)$: Advantage

Practical Implementation

Use parameters **before each model update step** as π_{prox} . Recompute token probabilities upon arrival of global batch in each training step.



Choosing the Right Staleness Budget

Empirical Findings

Experimental results demonstrate that **moderate staleness ($\eta \leq 8$)** has minimal impact on final performance while significantly accelerating training.

Key Observation

When properly constrained, moderate staleness maintains performance while enabling **2x+ speedup** through the asynchronous pipeline.

Empirical Recommendations

Coding Tasks

$\eta = 4$

More sensitive to staleness

Math Tasks

$\eta = 8$

More tolerant of staleness

Trade-off Consideration

When η is **too small**, generation throughput can be slowed down when extremely long trajectories are being generated. Therefore, adopting a **larger staleness-control parameter** is suggested for best system throughput.

Performance (Evaluation Scores) vs. Staleness

Max.Stale.	AIME24		AIME25		AMC23		MATH 500	
	W/o	With	W/o	With	W/o	With	W/o	With
0 (Oracle)	42.0		32.9		84.4		89.2	
1	41.8	42.1	30.7	31.9	83.3	85.2	89.9	89.8
2	40.0	41.8	32.1	32.5	82.3	84.3	89.6	89.6
4	23.3	42.2	23.1	32.0	58.5	85.1	66.9	89.5
8	35.7	41.0	27.8	31.1	81.2	82.9	87.8	89.2
16	35.8	38.7	26.2	32.5	78.4	83.2	87.4	89.1
∞	34.0	36.9	26.9	29.9	79.4	81.0	87.1	88.1

Practical Insight: The system-wide practice of adopting larger η motivates the need for an enhanced algorithm that can make effective use of more stale data.

Dynamic Microbatch Allocation

Handling Variable-Length Sequences

AReAL employs a **padding-free sequence packing strategy** with a dynamic allocation algorithm that balances token distribution across micro-batches.

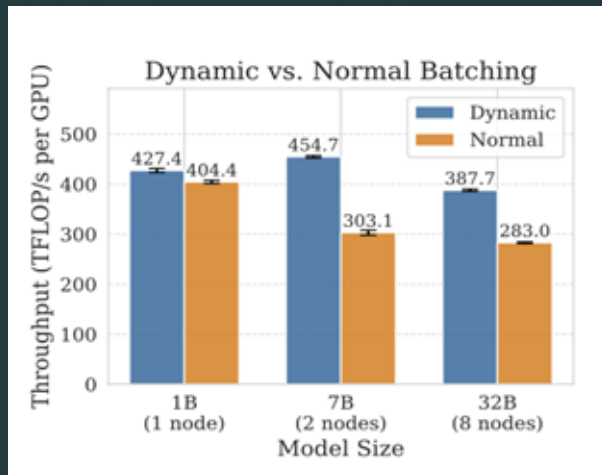
The Problem with Standard Batching

- ✗ Multiple long sequences in same micro-batch
- ✗ Requires many micro-batches to prevent OOM
- ✗ Inefficient memory utilization

Dynamic Batching Algorithm

1. Sort sequences by length (descending)
2. Pack into micro-batches under token budget
3. Balance distribution to maximize utilization
4. Minimize number of forward-backward passes

Performance Improvement



Interruptible Generation Gains

Eliminating the Generation Bottleneck

Without interruptible generation, the controller must wait for the **longest response** in each batch before weight updates can be applied.

The Difference

Non-interruptible

Wait for all sequences to complete

Interruptible

Update weights immediately when ready

Why It Matters

- ✓ **Reduces idle time:** Workers don't wait for slow sequences
- ✓ **Faster iteration:** More frequent weight updates
- ✓ **Better utilization:** Generation and training overlap

Experimental Results

Model	AIME24 ↑	# Nodes	PPO Steps	Training Hours ↓
1.5B basemodel	29.3	-	-	-
w/ VeRL	43.1*	16	250	33.6
w/ Sync.AReaL	42.0	16	250	41.0
w/ AReaL (ours)	42.2	16	250	14.8
7B basemodel	54.3	-	-	-
w/ VeRL	-	24	250	52.1
w/ Sync.AReaL	63.0	24	250	57.7
w/ AReaL (ours)	63.1	24	250	25.4

Model	LiveCodeBench ↑	# Nodes	PPO Steps	Training Hours ↓
14B basemodel	53.4	-	-	-
w/ VeRL	57.9*	32	80	44.4
w/ Sync.AReaL	56.7	32	80	48.8
w/ AReaL (ours)	58.1	32	80	21.9
32B basemodel	57.4	-	-	-
w/ VeRL	-	48	60	46.4
w/ Sync.AReaL	61.2	48	60	51.1
w/ AReaL (ours)	61.0	48	60	31.1

Near-Linear Scaling

Scalability to 512 GPUs

AReAL demonstrates **approximate linear scaling** with increased device count, while synchronous systems typically fail to scale effectively or crash.

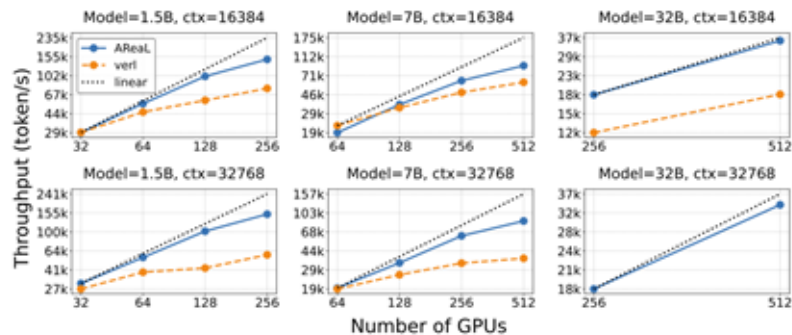
Scaling Efficiency

- ✓ Scales effectively up to **512 GPUs**
- ✓ Synchronous systems fail or encounter OOM
- ✓ Up to **2.5× throughput** advantage

Context Length Robustness

- 🛡️ **16K context:** Both systems work, AReAL faster
- 🛡️ **32K context:** verl encounters OOM, AReAL works
- 🛡️ **32B model + 32K:** verl fails completely

Scaling Comparison



Critical Advantage: AReAL's asynchronous design enables scaling where synchronous systems fail, making large-scale LRM training practical.

Strengths



Robustness to Long Contexts

AReaL handles **32K context lengths** where synchronous systems fail or encounter OOM errors. The asynchronous design naturally accommodates variable-length sequences.

Supported Context

Up to 32K+



Near-Linear Scaling

Demonstrates **approximate linear scaling** up to 512 GPUs, while synchronous systems fail to scale effectively. Makes large-scale training practical.

Maximum GPUs

512+



Significant Speedup

Achieves **up to 2.77x speedup** without performance degradation. In some cases, even improves final accuracy compared to synchronous training.

Maximum Speedup

2.77x

Weaknesses & Limitations



Fixed Resource Partitioning

Currently uses a **fixed 75/25 ratio** between inference and training devices. This ratio may not be optimal for all training scenarios and workload characteristics.

Current Configuration

75% Inference / 25% Training



Staleness Budget Tuning

Optimal staleness budget **varies by task type** ($\eta=4$ for coding, $\eta=8$ for math) and requires empirical tuning. No automatic adaptation mechanism.

Tuning Required

Task-Dependent



Single-Turn Focus

Currently focused on **single-turn reasoning tasks** (math, coding). Multi-turn interactions and agentic scenarios are not extensively explored.

Supported Tasks

Single-Turn Only



System Complexity

Asynchronous coordination introduces **additional system complexity** and overhead. Debugging and monitoring are more challenging than synchronous systems.

Complexity Level

Higher



Limited Task Diversity

Evaluation focused on **math and coding tasks**. Generalization to broader domains (creative writing, open-ended dialogue) is unclear.

Tested Domains

Math, Coding

Future Research Directions



Dynamic GPU Allocation

Automatically adjust inference/training ratio based on workload characteristics during training. Context lengths typically increase during RL, requiring dynamic rebalancing.

Impact

Optimal Resource Utilization



Multi-Turn Interactions

Extend AReaL to **agentic scenarios** with tool use, conversation, and multi-step reasoning. Explore applications in dialogue systems and interactive agents.

Impact

Broader Applicability



Heterogeneous Hardware

Support for **different GPU types** and distributed environments. Enable training across cloud providers, on-premise clusters, and edge devices.

Impact

Deployment Flexibility



Broader Task Domains

Apply AReaL to **general instruction following**, creative tasks, and open-ended generation. Evaluate effectiveness beyond reasoning benchmarks.

Impact

Generalization



Automatic Staleness Adaptation

Develop mechanisms to **automatically adjust** staleness budget based on training dynamics, eliminating manual tuning requirements.

Impact

Ease of Use